

Problema Lab 2: Tranzacții și Niveluri de Izolare

Obiectiv

Înțelegerea conceptelor de tranzacție a bazei de date prin implementarea și demonstrarea diferitelor niveluri de izolare a tranzacțiilor, probleme de concurență și optimizări de procesare în lot (batch).

Obiective de Învățare

- Înțelegerea proprietăților ACID în practică
- Experimentarea problemelor comune de concurență (dirty reads, non-repeatable reads, phantom reads)
- Învățarea nivelurilor de izolare a tranzacțiilor și compromisurile lor
- Optimizarea operațiilor în masă folosind tranzacții și batch processing
- Înțelegerea impactului asupra performanței al strategiilor de gestionare a tranzacțiilor

Cerințe

1. Demonstrații ale Nivelurilor de Izolare a Tranzacțiilor

Creați demonstrații pentru fiecare problemă de concurență. Fiecare demonstrație trebuie să:

- Folosească **două conexiuni separate la baza de date** simulând utilizatori concurenți
- Arate problema apărând la un nivel de izolare permisiv
- Arate problema prevenită la un nivel de izolare mai strict
- Înregistreze/afișeze pas cu pas ce face fiecare tranzacție

A. Demonstrație Dirty Read

Scenariu: Tranzacția A actualizează o înregistrare dar nu a făcut commit. Tranzacția B citește datele necomise.

Tranzacția A:

1. BEGIN TRANSACTION
2. UPDATE employees SET salary = 10000 WHERE id = 1
3. [AȘTEPTĂ - nu face commit încă]
4. Afișează: "Tranzacția A: Salariu actualizat la 10000 (ne-comis)"

Tranzacția B (rulează concurrent):

1. BEGIN TRANSACTION (READ UNCOMMITTED)
2. SELECT salary FROM employees WHERE id = 1
3. Afișează valoarea salariului citită
4. COMMIT

Tranzacția A (continuă):

5. ROLLBACK
6. Afișează: "Tranzacția A: Rollback efectuat!"

Întrebare pentru raport: Ce valoare a citit Tranzacția B? Ce nivel de izolare previne acest lucru?

B. Demonstrație Non-Repeatable Read

Scenariu: Tranzacția A citește o înregistrare de două ori. Tranzacția B actualizează înregistrarea între cele două citiri.

Tranzacția A:

1. BEGIN TRANSACTION (READ COMMITTED)
2. SELECT salary FROM employees WHERE id = 1
3. Afișează: "Prima citire: [valoare]"
4. [AȘTEPTĂ ca Tranzacția B să se finalizeze]
5. SELECT salary FROM employees WHERE id = 1
6. Afișează: "A doua citire: [valoare]"
7. COMMIT

Tranzacția B (rulează între citirile lui A):

1. BEGIN TRANSACTION
2. UPDATE employees SET salary = 12000 WHERE id = 1
3. COMMIT
4. Afișează: "Tranzacția B: Actualizat și comis"

Întrebare: Cele două citiri au returnat aceeași valoare? Ce nivel de izolare previne acest lucru?

C. Demonstrație Phantom Read

Scenariu: Tranzacția A execută aceeași interogare de două ori. Tranzacția B inserează un rând care corespunde interogării între execuții.

Tranzacția A:

1. BEGIN TRANSACTION (REPEATABLE READ)
2. SELECT COUNT(*) FROM employees WHERE department_id = 5
3. Afișează: "Prima numărătoare: [valoare]"
4. [AȘTEPTĂ Tranzacția B]
5. SELECT COUNT(*) FROM employees WHERE department_id = 5
6. Afișează: "A doua numărătoare: [valoare]"
7. COMMIT

Tranzacția B:

1. BEGIN TRANSACTION
2. INSERT INTO employees (name, department_id) VALUES ('Angajat Nou', 5)
3. COMMIT
4. Afișează: "Tranzacția B: Angajat nou inserat"

D. Demonstrație Lost Update

Scenariu: Două tranzacții citesc aceeași valoare, o modifică și o scriu înapoi. O actualizare se pierde.

Tranzacția A:

1. BEGIN TRANSACTION
2. SELECT salary FROM employees WHERE id = 1 // citește 5000
3. newSalary = salary + 1000 // calculează 6000
4. [AȘTEPTĂ]
5. UPDATE employees SET salary = 6000 WHERE id = 1
6. COMMIT

Tranzacția B (concurentă):

1. BEGIN TRANSACTION
2. SELECT salary FROM employees WHERE id = 1 // citește 5000
3. newSalary = salary + 500 // calculează 5500
4. UPDATE employees SET salary = 5500 WHERE id = 1
5. COMMIT

Cerințe de Implementare:

- Creați o aplicație GUI sau consolă cu butoane pentru a rula fiecare demonstrație
- Folosiți threading sau operații async pentru a simula tranzacții concurente
- Adăugați declarații sleep/wait pentru a controla timing-ul și a face problemele vizibile
- Afișați pas cu pas ce face fiecare tranzacție
- Arătați starea finală a bazei de date după fiecare demonstrație

2. Demonstrație Deadlock

Creați un scenariu care produce un deadlock:

Tranzacția A:

1. BEGIN TRANSACTION
2. UPDATE employees SET salary = 6000 WHERE id = 1 // blochează rândul 1
3. [AȘTEAPTĂ 2 secunde]
4. UPDATE employees SET salary = 7000 WHERE id = 2 // încearcă să blocheze rândul 2

Tranzacția B (concurentă):

1. BEGIN TRANSACTION
2. UPDATE employees SET salary = 6000 WHERE id = 2 // blochează rândul 2
3. [AȘTEAPTĂ 2 secunde]
4. UPDATE employees SET salary = 7000 WHERE id = 1 // încearcă să blocheze rândul 1

Cerințe:

- Implementați detectarea și gestionarea deadlock-urilor
- Arătați mesajul de eroare de deadlock
- Demonstrați rezolvarea cu ordonare corectă a tranzacțiilor

3. Comparație Performanță Inserare în Batch

Comparați performanța inserării a **5000 de înregistrări** folosind doua abordări diferite:

Comparați performanța inserării a **5000 de înregistrări** folosind trei abordări diferite:

Abordarea 1: Auto-commit (o tranzacție per inserare)

// Exemplu Java

```
try (Connection conn = DriverManager.getConnection(...)) {
    String sql = "INSERT INTO employees (name, ...) VALUES (?, ...)";

    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        for (int i = 0; i < 5000; i++) {
            stmt.setString(1, "Employee" + i);
```

```

        // Setează ceilalți parametri aici (dacă există)
        stmt.executeUpdate(); // Auto-commit implicit
    }
}

```

Abordarea 2: Commit-uri în loturi (commit la fiecare 100 de inserări)

```

try (Connection conn = DriverManager.getConnection(...)) {
    conn.setAutoCommit(false); // Dezactivează auto-commit
    String sql = "INSERT INTO employees (name, ...) VALUES (?, ...)";

    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        for (int i = 0; i < 5000; i++) {
            stmt.setString(1, "Employee" + i);
            // Setează ceilalți parametri aici (dacă există)
            stmt.executeUpdate();

            if (i % 100 == 0) {
                conn.commit(); // Commit la fiecare 100 de inserări
            }
        }
        conn.commit(); // Commit final pentru rândurile rămase
    } catch (SQLException e) {
        conn.rollback(); // Anulează în caz de eroare
        throw e;
    }
}

```

Abordarea 3: Tranzacție unică (toate cele 5000 într-o tranzacție)

```

try (Connection conn = DriverManager.getConnection(...)) {
    conn.setAutoCommit(false); // Tranzacție unică
    String sql = "INSERT INTO employees (name, ...) VALUES (?, ...)";

    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        for (int i = 0; i < 5000; i++) {
            stmt.setString(1, "Employee" + i);
            // Setează ceilalți parametri aici (dacă există)
            stmt.addBatch(); // Adaugă în lot

            if (i % 50 == 0) {
                stmt.executeBatch(); // Executa lotul la fiecare 50 de inserări
            }
        }
        stmt.executeBatch(); // Executa lotul rămas (dacă există)
        conn.commit(); // Commit final
    } catch (SQLException e) {
        conn.rollback(); // Anulează în caz de eroare
        throw e;
    }
}

```

Cerințe:

- Măsurați timpul de execuție pentru fiecare abordare (în milisecunde)
- Afișați rezultatele într-un tabel sau grafic
- Curățați (ștergeți înregistrările de test) între rulări
- Rulați fiecare abordare cel puțin de 3 ori și calculați media
- Includeți rezultatele și analiza în raport

4. Livrabile

1. **Cod sursă** cu toate demonstrațiile implementate
2. **Rezultate măsurări performanță** într-un format structurat (tabel/spreadsheet):
 - Comparație inserare în lot pentru toate abordările
 - Rulări multiple cu medii
3. **Raport detaliat (3-4 pagini)** acoperind:
 - Explicația fiecărei probleme de concurență demonstrate
 - Capturi de ecran sau log-uri pentru fiecare demonstrație
 - Analiza nivelurilor de izolare: când să folosiți fiecare
 - Analiza performanței inserărilor în lot cu concluzii
 - Compromisuri între performanță și consistența datelor
 - Scenarii din lumea reală unde fiecare nivel de izolare este potrivit

5. Criterii de Evaluare (100 puncte)

- **Demonstrații Concurență (40 puncte)**
 - Toate cele patru probleme demonstrate corect (20)
 - Logging/afișare clară pas cu pas (10)
 - Soluții corecte cu niveluri de izolare (10)
- **Testare Performanță Batch (30 puncte)**
 - Toate abordările implementate corect (15)
 - Măsurători precise cu rulări multiple (10)
 - Prezentare clară a rezultatelor (5)
- **Calitatea Codului (15 puncte)**
 - Gestionare corectă a tranzacțiilor (10)
 - Implementare curată a concurenței (5)
- **Raport (15 puncte)**
 - Analiză completă a problemelor de concurență (8)
 - Analiză performanță și concluzii (7)

6. Puncte Bonus (+10)

- Creați un grafic vizual de comparație pentru performanța inserării în lot

Predare

- **Termen limită:** Următoarea sesiune de laborator (14 de zile)